UNIVERSIDAD TECNOLÓGICA DEL VALLE DE TOLUCA

Dirección de Carrera de Tecnologías de la Información y Comunicación

Ingeniería en Tecnologías de la Información

Área Desarrollo y Gestión de Software

DESARROLLO MÓVIL INTEGRAL

Actividad 20 Integración de almacenamiento en la nube para archivos multimedia

Nombre del estudiante:

Lara Rodriguez Joel Alejandro

Grupo

IDGS – 101

Cuatrimestre

10º

Lugar

Santa María Atarasquillo, Lerma, México.

Fecha de Entrega

7 de noviembre de 2026

Almacenamiento privado de fotografías de incidencias

Actividad 20 Nexo Incidencias

Lara Rodriguez Joel Alejandro
IDGS 101 | 10º cuatrimestre
Desarrollo Móvil Integral
Docente R. Vinicio Camacho
7 de noviembre de 2026

Propósito y resultado

Desarrollé Nexo Incidencias para registrar problemas en instalaciones y adjuntar una fotografía que permita reconocerlos. La aplicación utiliza React Native y Expo SDK 58, con Supabase para autenticar al usuario, almacenar archivos privados y conservar los datos de cada reporte. La imagen permanece en Storage; la base de datos guarda su referencia y los metadatos que la relacionan con la incidencia.

Se comprobó el flujo con el servicio real: seleccionar un PNG, visualizarlo, cargarlo, recuperar una URL temporal y mostrarlo nuevamente después de recargar la aplicación web. Las pruebas también verificaron el rechazo de accesos anónimos y de otro usuario. Las fotografías utilizadas son ejemplos sintéticos identificados en las capturas. La cámara y el cierre y reapertura en un teléfono físico quedan pendientes de comprobar.

Funciones adicionales

- Registro e inicio de sesión, reportes separados por propietario y sesión móvil en SecureStore.
- Ubicación escrita, seis categorías, cuatro prioridades, estados Abierta, En atención y Resuelta.
- Panel de estadísticas, buscador, filtros, archivo y restauración de reportes.
- Borrador de texto, caché de metadatos por usuario e historial de cambios en la nube.
- Indicadores de carga por fase, reintento de la misma incidencia y visor de imagen completa.

Entregables y ubicación

Proyecto: C:\10moCuatri\Vini\Actividad20-NexoIncidencias
Repositorio: https://github.com/JoelLara10/Actividad20-NexoIncidencias
El código de selección y carga está en src/api.js; las pantallas están en src/IncidentForm.js y src/IncidentDetail.js. Las reglas se entregan en supabase/migrations. Los resultados y capturas están en evidencias.

Preparación del servicio de almacenamiento

Seleccioné Supabase Storage por su integración con autenticación y reglas RLS. Se creó una organización dedicada y un proyecto gratuito. El plan no garantiza disponibilidad permanente y puede aplicar pausas o cuotas; no se habilitaron recursos pagados.

Elemento	Configuración implementada
Servicio y proyecto	Supabase Storage · Nexo Incidencias · kizwbajlravjjhnbpkhm
Espacio de almacenamiento	Bucket incidence-photos · privado · región us-east-1
Formatos permitidos	JPEG, PNG y WebP; application/pdf y SVG no permitidos
Tamaño máximo de la app y bucket	5 242 880 bytes por fotografía, equivalente a 5 MiB
Mecanismo de carga	SDK JavaScript con ArrayBuffer, petición HTTPS autenticada mediante JWT
Condiciones de acceso	Usuario autenticado y primera carpeta de ruta igual a auth.uid()
Visualización	URL firmada de 120 segundos; se solicita nuevamente al abrir el reporte

Configuración realizada

- Crear organización y proyecto mediante la sesión autorizada del propietario.
- Aplicar la migración SQL: tablas incidents e incident_events, índices, triggers y políticas por propietario.
- Crear el bucket privado con tipos MIME y tamaño restringidos en el servidor.
- Configurar solo la URL y la clave pública en el archivo local .env, excluido de Git.
- Comprobar carga real, integridad SHA256, acceso autorizado y rechazo de accesos externos.

Recursos y condiciones de uso

Las imágenes de prueba fueron generadas para esta actividad y no contienen fotografías personales. El uso del servicio está sujeto a las condiciones y cuotas de Supabase. La entrega identifica al proveedor y enlaza su documentación. No se incorporan claves administrativas al código móvil ni al repositorio.

Implementación y procesamiento de respuestas

Flujo de la aplicación

Iniciar sesión → escribir incidencia → seleccionar galería o cámara → validar formato y tamaño → mostrar vista previa → guardar metadatos pendientes → subir fotografía al bucket privado → confirmar estado listo → solicitar URL temporal → visualizar imagen remota.

Selección validación y transferencia

```
const result = await ImagePicker.launchImageLibraryAsync({
  mediaTypes: ['images'], allowsEditing: true, quality: .82, exif: false
});
const { bytes, mime } = await inspectPhoto(result.assets[0]);
const { error } = await supabase.storage.from('incidence-photos')
  .upload(path, bytes, { contentType: mime, upsert: true });
```

La selección puede cancelarse sin borrar el formulario. La cámara solicita permiso al tocar el botón. La app lee la fotografía como ArrayBuffer y valida firmas de JPEG, PNG o WebP, además del tamaño. Calcula SHA256 para identificar los bytes. La ruta combina UUID del usuario, UUID de la incidencia y parte del hash. Se conserva el UUID para reintentar sin duplicar el reporte.

Interpretación de JSON

El cliente Supabase convierte las respuestas JSON en objetos data y error. Las consultas de incidents entregan arreglos con título, ubicación, categoría, prioridad y estado. El inicio de sesión devuelve sesión y usuario. Storage devuelve la ruta del objeto; createSignedUrl devuelve data.signedUrl. Cada error se transforma en un mensaje visible. La fotografía se recibe como contenido binario; no forma parte del JSON de la incidencia.

```
const { data, error } = await supabase.storage
  .from('incidence-photos').createSignedUrl(photo_path, 120);
if (error) throw error;
// Image usa data.signedUrl como URI remota
```

Coordinación de archivo y reporte

La fila se registra primero con photo_state pending. Tras cargar el objeto se actualiza a ready; un trigger verifica que exista en storage.objects. Ante un error se mantiene el formulario y la transferencia se puede retomar. No existe una transacción atómica entre Storage y PostgreSQL: un corte puede dejar una referencia pendiente o un archivo ya cargado. Reintentar con el mismo ID y ruta permite completar el flujo; no hay cola automática en segundo plano.

Endpoints y datos que conserva la base de datos

Todas las peticiones remotas utilizan la base HTTPS https://kizwbajlravjjhnbpkhm.supabase.co y una clave pública de proyecto. Las operaciones privadas incorporan el JWT del usuario autenticado.

Método y ruta	Finalidad y datos
POST /auth/v1/signup	Correo y contraseña para registrar una cuenta; confirmación del correo.
POST /auth/v1/token?grant_type=password	Iniciar sesión y obtener JWT de usuario.
GET POST PATCH /rest/v1/incidents	Consultar, reservar o actualizar metadatos según RLS.
GET /rest/v1/incident_events	Obtener el historial de la incidencia propia.
POST /storage/v1/object/incidence-photos/{ruta}	Transferir ArrayBuffer con MIME y autorización del usuario.
POST /storage/v1/object/sign/incidence-photos/{ruta}	Solicitar JSON con enlace temporal; expiresIn 120.
GET al enlace temporal	Descargar los bytes y mostrarlos con Image.

Relación entre fotografía e incidencia

Datos persistentes	Uso
id y owner_id	Identificar el reporte y su propietario autenticado.
title description location category priority status	Describir el problema y su atención.
photo_path y photo_state	Localizar el archivo y conocer si está pendiente, falló o está listo.
photo_mime photo_bytes photo_sha256	Describir el archivo y verificar integridad.
created_at updated_at archived	Ordenar, rastrear cambios y archivar sin borrar la evidencia.

La tabla no guarda bytes, base64, contraseñas ni URLs firmadas. incident_events conserva ID del reporte, propietario, mensaje y fecha. Los borradores y los últimos cien metadatos se guardan localmente por usuario; la fotografía pendiente no se conserva al cerrar el proceso y debe seleccionarse otra vez.

Evidencias de selección y carga

Capturas de la aplicación real exportada a web, probada a 390 × 844 píxeles. Los servicios de autenticación, base de datos y almacenamiento son el proyecto Supabase real. El archivo de incidencia mostrado es una imagen sintética de prueba.

Fotografía seleccionada



Vista previa local, formato y tamaño antes de transferir.

Carga completada



Confirmación después de almacenar y relacionar la fotografía.

Evidencia de almacenamiento remoto

La siguiente captura muestra el resultado de una consulta real a storage.objects y storage.buckets mediante la API de administración autorizada. Identifica ruta, contenedor, tamaño, MIME y public false. Es una presentación de la consulta, no una captura del Dashboard. El JSON original se incluye en evidencias/objetos-cloud.json.



La prueba de descarga recuperó 20 853 bytes. El SHA256 del archivo recuperado coincide con el de la imagen seleccionada: c8ffd4af30ad599140a59ceaf80baa5a1a44ef899029205fb712864654eefecd.

Comprobación en el panel del servicio

Abrir https://supabase.com/dashboard/project/kizwbajlravjjhnbpkhm/storage/buckets y seleccionar incidence-photos. Los archivos están organizados por UUID de usuario y reporte. Mantener el contenedor privado; para visualizarlos desde la app se solicita una URL temporal con la sesión correspondiente.

Recuperación y comportamiento sin conexión

Fotografía recuperada



Después de recargar la aplicación se consultan la referencia y la imagen remota.

Error de transferencia



Corte de red controlado durante la carga; mensaje visible y datos conservados.

El corte se produjo bloqueando la petición de carga en el navegador de pruebas. No se fingió una respuesta exitosa. Al retirar el bloqueo, el mismo formulario completó una carga real. Además se inició una nueva sesión mediante la API y se verificó que la referencia y el estado listo persistían en PostgreSQL. La prueba física de cierre y reapertura en Expo Go debe completarse en el teléfono del alumno.

Pruebas documentadas y justificación

Caso y justificación	Procedimiento y resultado
P1 Selección y carga válida Confirma el flujo principal.	Elegir PNG de 20 853 bytes, completar campos y subir. Resultado: archivo real almacenado, estado ready y URL temporal con imagen visible. Aprobado.
P2 Recuperación persistente Descarta depender solo de la vista previa local.	Recargar la web y abrir el reporte; además iniciar sesión nuevamente mediante API. Resultado: misma referencia en la nube y fotografía recuperada. Aprobado en web; pendiente repetir en teléfono.
P3 Formato no permitido Evita usar Storage para archivos ajenos al objetivo.	Pruebas de bytes con PDF y SVG renombrados; envío directo de application/pdf al bucket. Resultado: app rechaza la firma y servidor rechaza el MIME. Aprobado.
P4 Pérdida de conexión Verifica que no se anuncie éxito inexistente.	Bloquear solo POST de carga en el navegador. Resultado: error visible, datos conservados y reintento real exitoso al restablecer la petición. Aprobado.
P5 Acceso privado Comprueba aislamiento entre usuarios.	Consultar sin autorización, usar URL pública y solicitar firma desde otra cuenta. Resultado: acceso denegado; la consulta de otra cuenta no devuelve la fila. Aprobado.
P6 Límites y validación Evita transferencias innecesarias.	Casos unitarios de vacío, tamaño superior a 5 MiB y campos inválidos. Resultado: rechazo antes de transferir. Aprobado.

Resultados: siete grupos de pruebas unitarias aprobados, siete comprobaciones contra Supabase aprobadas y cinco verificaciones del flujo de interfaz sin errores JavaScript. Se exportaron las plataformas web y Android. La exportación Android es el paquete JavaScript y recursos, no un APK instalado ni una prueba física.

Los registros reproducibles se encuentran en pruebas-cloud.json y prueba-interfaz.json. La evidencia de la desconexión está identificada como fallo simulado; las cargas y descargas posteriores utilizaron el servicio real.

Mecanismos de seguridad

Autenticación y autorización

Supabase Auth entrega JWT por usuario. RLS compara owner_id con auth.uid() en tablas y exige que la carpeta inicial del objeto corresponda al mismo usuario. No hay acceso anónimo a los reportes.

Transferencia y claves

HTTPS en todas las consultas remotas. El cliente contiene únicamente URL y clave publicable. service_role, claves secretas y contraseña de base de datos se excluyen del código, del documento y de Git.

Privacidad de archivos

Bucket privado y enlaces firmados de 120 segundos. Estos enlaces son credenciales temporales: quien los posea puede usarlos durante su vigencia; no se publican ni se guardan en la tabla. Se solicitan nuevamente al abrir la imagen.

Validación de archivos

Tipos JPEG, PNG y WebP y límite 5 MiB. La app revisa firma binaria; el servicio restringe MIME y tamaño. No incluye antivirus ni análisis de contenido en servidor; los límites MIME no sustituyen una inspección profunda de archivos.

Sesión y datos locales

En móvil la sesión se guarda en SecureStore, con fragmentos para manejar su tamaño; en web se usa sessionStorage. Los metadatos y borradores locales usan AsyncStorage sin cifrado adicional. Evitar texto sensible y rostros en las evidencias.

Errores y disponibilidad

Tiempo máximo de consulta de veinte segundos, indicadores por fase y mensajes visibles. Las consultas guardadas pueden verse como metadatos de caché; una fotografía remota requiere conectividad. El reintento conserva el identificador de incidencia.

Guía de ejecución y fuentes

Abrir el prototipo

```
cd C:\10moCuatri\Vini\Actividad20-NexoIncidencias
npm install
npx expo start --clear
```

La copia local ya tiene .env con datos públicos de conexión. En otra computadora se debe copiar .env.example y completar la URL y clave publicable del proyecto. Usar Expo Go compatible con SDK 58 y escanear el QR nuevo; cerrar servidores antiguos de SDK 57. Si la red local impide conectar, utilizar npx expo start --tunnel. Crear una cuenta propia y confirmar el correo. Las credenciales de las cuentas de prueba permanecen solo en archivos locales privados.

Demostración sugerida

- Iniciar sesión y mostrar el panel; crear un reporte con todos sus campos.
- Elegir una fotografía desde galería o cámara y mostrar la vista previa.
- Registrar la incidencia y explicar las cuatro fases de la carga.
- Abrir el reporte, mostrar la imagen remota y cambiar su estado.
- Cerrar y volver a abrir Expo Go en el teléfono para completar esa comprobación física.
- Presentar el archivo en el bucket privado y las evidencias de error y acceso denegado.

Verificaciones de código

```
npm test
npm run build:web
npx expo export --platform android --output-dir dist-android
```

Las pruebas cloud y de navegador están incluidas como scripts; requieren las credenciales de QA guardadas fuera de Git. No es necesario usar una clave administrativa para ejecutar la aplicación.

Documentación oficial consultada

Almacenamiento y buckets privados https://supabase.com/docs/guides/storage/buckets/fundamentals

Políticas RLS para objetos https://supabase.com/docs/guides/storage/security/access-control

Descargas y enlaces temporales https://supabase.com/docs/guides/storage/serving/downloads

Selección de imágenes y cámara https://docs.expo.dev/versions/latest/sdk/imagepicker/

Cuotas y planes https://supabase.com/docs/guides/platform/billing-on-supabase

Condiciones del servicio https://supabase.com/terms